

ServeFacialAttributes: Scalable Model Deployment System for Human Face Analysis

Abstract. ServeFacialAttributes is a scalable microservice built using TorchServe for facial attribute classification and regression tasks. It provides a flexible and easily expandable alternative to proprietary APIs, allowing the addition of new models to broaden its application scope for preprocessing images. The microservice includes pre-trained open-source models for face detection, head pose estimation, and classifiers for gender, emotions, race, and landmark detection. This paper describes the architecture and functionality of the ServeFacialAttributes project.

Keywords: facial attribute classification · deep learning · microservice

1 Introduction

Multi-model machine learning systems require orchestration of specialized components developed for distinct tasks. Deploying these systems involves integrating new models without architectural changes, scaling to handle concurrent inference requests, maintaining test coverage across all components, and managing conflicting dependencies between models.

Facial attribute analysis exemplifies these challenges, requiring detection, landmark extraction, attribute classification and other downstream tasks. Current solutions often rely on proprietary APIs like Microsoft Face API [7], which lack customizing support and deprecation risks.

The initial motivation for our project arose from the need to create annotated facial datasets for training conditional generative models. When the Microsoft Face API used in the FFHQ-UV dataset [1] work became deprecated, we faced the challenge of reproducing and extending the dataset’s attribute annotations. This highlighted the need for a sustainable, customized solution that would not be vulnerable to API deprecation or policy changes.

ServeFacialAttributes addresses these limitations by providing a template-based framework for deploying multiple models as microservices. Our contributions include:

1. Unified model deployment architecture with uniform interfaces
2. Template-driven handler development system
3. Testing pipeline for handler validation
4. Reference implementation with 15 facial attribute models

The template architecture works for any domain requiring multi-model orchestration, not limited to facial analysis, making it applicable to diverse machine learning tasks. We provide our code and documentation in Github repository¹.

2 Related Work

2.1 Model Serving Systems

TensorFlow Serving [3] provides deployment infrastructure for TensorFlow models. NVIDIA Triton Inference Server [4] allows to work with models from different frameworks, but require deep dive into NVIDIA ecosystem. MLflow [5] provides end-to-end tools for managing machine learning workflows including experiment tracking, reproducible project packaging, model versioning, and a registry for model deployment.

However, for serving multiple PyTorch models in production, TorchServe [2] provides a more convenient solution, offering multi-model deployment, and scalability.

2.2 Facial Analysis Systems

Commercial APIs from Microsoft, Amazon [8], and Google [9] provide proprietary facial analysis services with limited customization, making them difficult to adapt, hard to reproduce, and eventually deprecated. Open-source projects like OpenFace [10] offer specific facial analysis functions but in most cases it is needed to extend and customize. Our approach bridges this gap by providing both the models and a uniform deployment methodology.

3 System Design

3.1 Design Principles

The service is built in a way to make adding models, preparation for deployment and testing functionality of new models standardized and templated to speed up and simplify development. At the same time, each model is wrapped in a separate microservice, which makes models isolated and simplifies debugging during adding new models or updating already added ones and makes inspection in case of problems more convenient and comfortable. We also implemented testing modules and fixtures to cover all stages of model inference. Implementation details will be described below.

3.2 TorchServe Foundation

We use TorchServe as the underlying infrastructure framework for our system as far as it delivers native support for PyTorch models, provides REST API endpoints exposure through standardized interfaces and allows to customize model handlers at each step of data flow - preprocess, inference and postprocess.

¹ anonymous github

3.3 Template-Based Architecture

The system consists of a set of basic components. The system employs a base handler for model integration, while derived handlers inherit from this it, implementing the request-processing interface for the microservice. A packing script automatically packages the model archives for TorchServe. It bundles the handler with its dependencies into the archive, removing the need for manual setup. Each model is provided with a configuration file that specifies the links to its weights and defines the recommended hardware requirements (GPU or CPU). Deployment configs specify the number of workers and other TorchServe parameters.

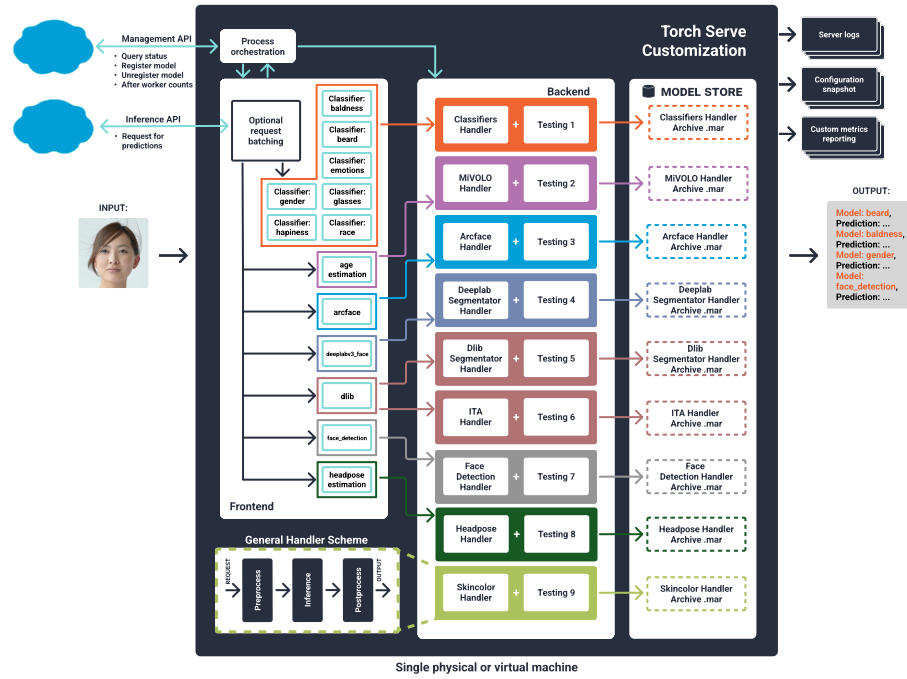


Fig. 1. ServeFacialAttributes API scheme: the picture shows core points of model deployment such as General Handler Scheme, Model Store and etc. and brief pipeline description.

4 Implementation Details

4.1 Handler Template

Model handlers consists of 4 functions: `initialize()`, `preprocess()`, `inference()`, `postprocess()`. Initialize function implements model and weight loading. Pre-

process stage contains functionality to extract content from request body and transform it into format which is compatible for model. During the inference stage model is applied to preprocessed data. In postprocess step the structured response generated from inference result - converts predictions to API output format. Data flow inside handler is depicted on Figure 1 in left below scheme.

4.2 Model Archive Generation

The model archive generation process packages the handler, model weights, and model dependencies into a deployable unit. Firstly, it verifies handler implementation completeness by running mock tests which will be described more detailed below. After that handler code and its dependencies and weights are packaged into model archive.

4.3 Testing Framework

In order to test handlers before wrapping them in a model archive, we mock the context of model for testing it before deployment. The context in TorchServe is an object that provides the information and methods needed for handling requests and maintaining models. It is used by handlers to access request data, TorchServe metadata, and other necessary resources. We implemented the computer vision domain specific fixtures and tests for 15 models listed below. In case of changing the domain it is required to adopt or add custom fixtures.

4.4 Implemented Models

Our reference implementation includes the following facial attribute analysis models:

1. **Face Detection:** YOLOv8-nano-based model [24] for identifying and localizing faces in images
2. **Head Pose Estimation:** Implementation of 6DRepNet360 [11] for precise head orientation calculation
3. **Age Estimation:** MiVOLO [12] regression model for accurate age prediction
4. **Gender Classification:** Binary ResNet50 [23] classifier trained on CelebA [18] and FairFace [19] datasets
5. **Emotion Recognition:** Seven-class ResNet50 classifier trained on FER2013 [20] and AffectNet [13] datasets
6. **Race Classification:** Seven-class ResNet50 classifier trained on the FairFace dataset for race analysis
7. **Beard Detection:** Binary ResNet50 classifier trained on CelebA dataset
8. **Baldness Detection:** Binary ResNet50 classifier trained on CelebA dataset
9. **Glasses Detection:** Binary ResNet50 classifier trained on CelebA and synthetic glasses dataset [21]

10. **Smile Detection:** Binary ResNet50 classifier for smile detection trained on emotion datasets
11. **Face Segmentation (Dlib [22]):** Traditional approach using landmark-based segmentation
12. **Face Segmentation (DeepLabV3):** Semantic segmentation using DeepLabV3+ [14] trained on CelebAMask-HQ [15]
13. **Skin Color Analysis:** Implementation based on the multidimensional measure of apparent skin color [26]
14. **Individual Typology Angle:** Calculator for skin tone classification based on colorimetric approach [17]
15. **Face Embeddings:** ArcFace [16] implementation for face verification and identification

5 Discussion & Limitations

The main limitation of our system lies in its reliance on Nox [25] during the testing phase, particularly when handling multiple requirement variants: in case of our facial service MiVOLO model has separate requirements. Nox is tool for automating testing in multiple Python environments. Large number of independent environments can lead to linear increase of the time required for testing all models.

Due to privacy consideration our service does not collect input data or processed out in any form. We do not provide any database or warehouse in our pipeline.

6 Conclusion & Future work

ServeFacialAttributes demonstrates a template-based approach for deploying multiple models as microservices. Our architecture reduces development effort, improves maintainability. While applied to facial attribute analysis, the toolkit is generally applicable to multiple model systems.

Future work includes application to other domains beyond computer vision.

References

1. Bai, Y., Kushwaha, V., Agarwal, S., Agarwal, S., Garces, E., Lazova, V., Weiss, T., Kanazawa, A., Belongie, S., Kemelmacher-Shlizerman, I.: FFHQ-UV: Normalized Facial UV Texture Dataset for 3D Face Reconstruction. 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 362-371 (2023)
2. PyTorch Serving Team. TorchServe: Serve, optimize and scale PyTorch models in production. Available: <https://github.com/pytorch/serve>. Accessed: 2025-03-27
3. Olston, C., Fiedel, N., Gorovoy, K., Harmsen, J., Lao, L., Li, F., Rajashekhar, V., Ramesh, S., Soyke, J.: TensorFlow-serving: Flexible, high-performance ML serving. In: Workshop on ML Systems at NIPS 2017 (2017)

4. NVIDIA: Triton Inference Server. <https://github.com/triton-inference-server/server>. Accessed: 2025-03-27
5. Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S.A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., et al.: Accelerating the machine learning lifecycle with MLflow. In: 2018 IEEE International Conference on Data Mining Workshops (ICDMW). pp. 1777–1777. IEEE (2018)
6. Seldon Core: Deploy, Manage, and Monitor Machine Learning Models in Kubernetes. <https://github.com/SeldonIO/seldon-core>. Accessed: 2025-03-27
7. Microsoft Azure Face API. <https://azure.microsoft.com/en-us/products/ai-services/ai-vision>. Accessed: 2025-03-27
8. Amazon Rekognition. <https://aws.amazon.com/rekognition/>. Accessed: 2025-03-27
9. Google Cloud Vision API. <https://cloud.google.com/vision/>. Accessed: 2025-03-27
10. Amos, B., Ludwiczuk, B., Satyanarayanan, M., et al.: OpenFace: A general-purpose face recognition library with mobile applications. CMU-CS-16-118, CMU School of Computer Science, Tech. Rep. (2016)
11. Hempel, T., Abdelrahman, A.A., Al-Hamadi, A.: Toward Robust and Unconstrained Full Range of Rotation Head Pose Estimation. *IEEE Transactions on Image Processing*, 33, 2377–2387 (2024)
12. Kuprashevich, M., Tolstykh, I.: MiVOLO: Multi-input Transformer for Age and Gender Estimation. In: *International Conference on Analysis of Images, Social Networks and Texts*. pp. 212–226. Springer (2023)
13. Mollahosseini, A., Hasani, B., Mahoor, M.H.: AffectNet: A Database for Facial Expression, Valence, and Arousal Computing in the Wild. *IEEE Transactions on Affective Computing*, 10(1), 18–31 (2019)
14. Chen, L.C., Zhu, Y., Papandreou, G., Schroff, F., Adam, H.: Encoder-decoder with atrous separable convolution for semantic image segmentation. In: *Proceedings of the European conference on computer vision (ECCV)*. pp. 801–818 (2018)
15. Lee, C.H., Liu, Z., Wu, L., Luo, P.: MaskGAN: Towards Diverse and Interactive Facial Image Manipulation. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2020)
16. Deng, J., Guo, J., Xue, N., Zafeiriou, S.: Arcface: Additive angular margin loss for deep face recognition. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. pp. 4690–4699 (2019)
17. Phung, S.L., Bouzerdoum, A., Chai, D., et al.: A novel skin color model in YCbCr color space and its application to human face detection. In: *Proceedings. International Conference on Image Processing*. vol. 1, pp. I–I. IEEE (2002)
18. Liu, Z., Luo, P., Wang, X., Tang, X.: Large-scale celebfaces attributes (celeba) dataset. Retrieved August, 15, 11 (2018)
19. Karkkainen, K., Joo, J.: Fairface: Face attribute dataset for balanced race, gender, and age for bias measurement and mitigation. In: *Proceedings of the IEEE/CVF winter conference on applications of computer vision*. pp. 1548–1558 (2021)
20. Khairuddin, Y., Chen, Z.: Facial emotion recognition: State of the art performance on FER2013. *arXiv preprint arXiv:2105.03588* (2021)
21. Heaton, J.: Glasses or No Glasses: A Binary Classification Dataset for Computer Vision. Kaggle (2023). <https://www.kaggle.com/datasets/jeffheaton/glasses-or-no-glasses>. Accessed: 2025-03-27
22. King, D.E.: Dlib-ml: A Machine Learning Toolkit. *Journal of Machine Learning Research*, 10, 1755–1758 (2009)

23. He, K., Zhang, X., Ren, S., Sun, J.: Deep Residual Learning for Image Recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016)
24. Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. Version 8.0.0, 2023. Available: <https://github.com/ultralytics/ultralytics>. Accessed: 2025-03-27
25. Alethea K. Flowers, "Nox: Flexible Test Automation with Python", 2016. [Online]. Available: <https://nox.thea.codes/>. Accessed: 2025-03-27
26. William Thong, Przemyslaw Joniak, and Alice Xiang. "Beyond Skin Tone: A Multidimensional Measure of Apparent Skin Color." In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 4903-4913.